

BR-Bench: A Benchmark Linking Continuous Integration Changes to Fault-Revealing Tests

Deepanshu*, Sanskriti Singh*, Prisha Vadhavkar*, Dr. Yoga Raja C A[†]

*School of Computer Science Engineering and Information Systems

Vellore Institute of Technology, Vellore, Tamil Nadu, India

{[EMAIL], [EMAIL], [EMAIL]}

[†]Assistant Professor Sr. Grade 1, Department of Information Technology

School of Computer Science Engineering and Information Systems

Vellore Institute of Technology, Vellore, Tamil Nadu, India

yogaraja.ca@vit.ac.in

Abstract—Continuous Integration (CI) services execute automated test suites to detect regressions introduced by pull requests. While industrial platforms at Meta and Google leverage predictive models and large-scale workload analyses, the research community lacks public, reproducible benchmarks linking pull-request modifications directly to individual failing tests. We present BR-Bench, an open dataset linking code changes to fault-revealing tests mined from GitHub Actions logs across Java and Python open-source repositories. A test is defined as fault-revealing if and only if it fails at the head commit, passes or is absent in the resolved base run, and does not exhibit flakiness across runs on the same commit. The strict benchmark split contains 762 pull-request instances, 4,168 fault-revealing labels, and 2,466 distinct tests across 37 repositories. We evaluate test-to-file binding against pinned commit trees, achieving 94.29% (5,643/5,985) combined binding and 63.83% (3,820/5,985) full-confidence fully qualified name binding. In an illustrative application evaluating predictive test selection proxies at $k = 10$, historical failure frequency achieves a micro recall of 39.44% (519/1,316) whereas all-partner co-change file mining achieves only 4.64% (61/1,316). We provide complete extraction pipelines without language model dependencies, processed release tables, and offline reproduction via a single command.

Index Terms—continuous integration, test selection, regression testing, mining software repositories, fault revelation

I. INTRODUCTION

Modern software development relies on continuous integration (CI) workflows to catch defects before pull requests merge into shared branches. In CI pipelines, every code modification triggers test suites that consume substantial computational time and energy. Predicting in advance which individual tests are likely to fail on a given pull-request diff would enable teams to run failure-prone tests first, accelerate feedback cycles, and conserve compute resources.

Prior academic research addresses neighboring questions rather than empirical test-failure prediction. Change impact analysis (CIA) techniques predict co-changed program entities using version-control history [1], [2], capturing developer editing coupling rather than runtime defect propagation. Regression test selection (RTS) tools, such as Ekstazi [3] and STARTS [4], compute safe over-approximations of reachable tests via dynamic file or static class dependencies. In

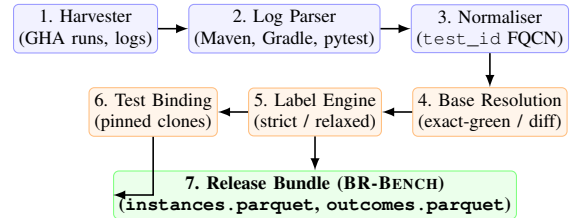


Fig. 1. The BR-BENCH construction pipeline. Raw CI logs and metadata are harvested, parsed into canonical test identifiers, matched against resolved base runs, filtered for flakiness, bound to source trees, and emitted as Parquet tables.

contrast, industrial studies examine CI workloads and predictive selection: Meta deployed predictive test selection on historical execution outcomes [5], while Google characterized continuous testing workloads and test selection trade-offs at massive scale [6]. However, these industrial deployments run on closed-source monorepos, leaving their underlying failure datasets inaccessible to academic researchers.

Existing open datasets fail to fill this gap. RTPTorrent [7] was mined from Travis CI prior to 2020, covers only Java, organizes data around build jobs rather than pull-request diffs, and emphasizes test prioritization rather than change-to-test fault linking. GHALogs [8] provides raw GitHub Actions logs but does not parse granular test outcomes, map tests to source files, or filter non-deterministic executions.

To bridge this divide, we present BR-BENCH, an open benchmark connecting pull-request file changes directly to the individual test cases that failed because of them. Mined from real GitHub Actions pipelines across 76 swept repositories, BR-BENCH extracts structured test outcomes from Gradle, Maven, and pytest execution logs. We resolve base runs on repository histories, filter same-commit test flakiness, and bind test identifiers back to repository source paths.

II. DATA SOURCE AND COLLECTION METHOD

The construction pipeline of BR-BENCH consists of seven reproducible stages (Fig. 1): repository selection, metadata harvesting, log parsing, identifier canonicalisation, base-run resolution, test-to-file binding, and release bundling.

A. Repository Sampling and Harvesting

We selected candidate repositories from the SEART GitHub platform [9]. Starting from an export of 3,671 open-source repositories, we retained 2,646 repositories active on GitHub Actions with at least 100 workflow runs in the preceding 90 days. We identified 2,333 repositories executing test-intent workflows (excluding linting or deployment tasks). We sampled 300 repositories into a balanced evaluation frame across Java and Python. The funnel counts 76 swept repositories and 165,349 runs (`instances_raw`); the harvest corpus has 77 repositories and 175,038 runs (`corpus_stats.md`). For each workflow run, the harvester downloaded run metadata, job step statuses, check-run annotations, and raw textual execution logs via the GitHub REST API.

B. Log Parsing and Test Identifier Normalisation

Because GitHub Actions treats test execution as unstructured stdout and stderr streams, test verdicts must be parsed from job logs. We built deterministic line parsers for three major test runners: Maven Surefire/Failsafe, Gradle Test Runner, and pytest. The parsers extract test outcomes (pass, fail, error, skip) and execution durations without relying on external large language models (LLMs). Each parsed test is mapped to a canonical identifier (`src/parse/test_ids.py`): for Java, the format is `fully.qualified.Class#method`; for Python, the format is `path::name`. Bare class identifiers are disambiguated using package declarations recovered from logs where available.

C. Base-Run Resolution and Fault Revelation

A test failure at a pull request’s head commit does not imply that the pull request caused the failure; the test may have been broken in the upstream base branch. To establish causality, every failed head run must be paired with an appropriate base run. For each pull-request run on head commit H targeting base branch commit B , we search repository history for a workflow run on B executing the identical workflow. If a run on B exists and completed with conclusion success, we record an `exact_green` resolution. If no run exists on B , we traverse ancestor commits along the default branch within a maximum distance of 10 commits. In total, base resolution records five mutually exclusive statuses: `exact`, `exact_green`, `ancestor` (max distance 10), `branch_prior`, and `no_base`. Under Invariant 6, runs resolving to `no_base` emit zero labels.

The benchmark defines three nested splits (`src/label/fault_revealing.py`): (1) `all` contains every (run, test) head failure in workflow runs with a usable base and parsed head test output; (2) `relaxed` removes tests that also fail in the resolved base run ($T_{\text{head_fail}} \setminus T_{\text{base_fail}}$); and (3) `strict` equals relaxed minus same-SHA flips (46/4,214 labels removed, 1.09%) to prune non-deterministic flakiness (T_{flaky}), yielding 4,168 strict positive labels:

$$T_{\text{reveal}} = (T_{\text{head_fail}} \setminus T_{\text{base_fail}}) \setminus T_{\text{flaky}}. \quad (1)$$

TABLE I
KEY SCHEMA FIELDS OF BR-BENCH RELEASE TABLES

Table	Field	Type and Description
instances	instance_id	String; PK: sha256(repo head run)
	repo	String; repository (owner/name)
	language	String; language (java or python)
	head_sha	String; commit SHA at head
	base_sha	String; target commit on base branch
	base_run_id	Int64; resolved base run identifier
	run_id	Int64; workflow run identifier
	workflow_name	String; CI workflow name
	author_login	String; HMAC-SHA256 pseudonymised hash
	changed_files	[VERIFY: in spec, omitted from release parquet]
	base_run_dist	[VERIFY: in spec, in base_resolution_new]
outcomes	run_id	Int64; FK to instances
	test_id	String; canonical test identifier
	split	String; split (all, relaxed, strict)
	status_head/base	[VERIFY: spec defines pass fail error skip absent]
	binding_strategy	[VERIFY: spec defines tests tests_by_convention tests_by_layout unbound]
cochange	file_a, file_b	String; pair of co-changed files
	support, lift	Int64 / Double; co-change metrics

D. Flakiness Filtering and Test-to-File Binding

Flaky tests flip their status non-deterministically without code changes [10]. We detect same-SHA flips across repeated workflow executions on the exact same commit hash. Across 11,557 evaluated (head SHA, workflow, test) groups, 62 instances (0.54%) flipped, leading to the removal of 46 flaky labels from the relaxed set (1.09% of 4,214 labels) to form the strict set.

To enable change impact analysis, canonical test identifiers must bind to physical test source files. We bind identifiers against pinned repository clones recorded in `docs/CLONE_PINS.json` rather than volatile branch heads. Basename-only matches achieve 0.5 confidence, while fully qualified class names that match uniquely achieve 1.0 confidence.

III. STORAGE AND SCHEMA

BR-BENCH is published in Apache Parquet format to ensure efficient columnar storage and typed schema enforcement, accompanied by a structured dataset datasheet [11]. Table I details the primary fields across the shipped release tables. The `instances.parquet` table stores pull-request runs, commit hashes, base-run metadata, and execution details. The `outcomes.parquet` table records individual test failures across the benchmark splits. The `cochange.parquet` table provides commit-level co-change association rules, and `failure_messages.parquet` isolates textual failure messages. Where fields from the design specification (`docs/SCHEMAS.md`) are resolved in upstream interim tables or evaluated in dedicated reports rather than emitted directly into release Parquet columns, they are marked [VERIFY].

To protect contributor privacy, all author usernames in `instances.parquet` are pseudonymised using HMAC-SHA256 keyed by an operator-held secret, truncated to 16 hex characters across 1,791 unique authors

TABLE II
PIPELINE ATTRITION FUNNEL ACROSS STAGES

Attrition Stage	Count	% Prev.	% Total
<i>Repositories</i>			
SEART GitHub export	3,671	—	100.00%
CI-live (≥ 100 runs in 90 days)	2,646	72.08%	72.08%
Test-intent workflow detected	2,333	88.17%	63.55%
Sampled into candidate frame	300	12.86%	8.17%
Swept by harvester	76	25.33%	2.07%
Repositories with ≥ 1 failed run	69	90.79%	1.88%
Repositories with ≥ 1 strict instance	37	53.62%	1.01%
<i>Workflow Runs</i>			
Total runs discovered	165,349	—	100.00%
Failed runs	12,581	7.61%	7.61%
Failed-job log captured on disk	6,586	52.35%	3.98%
Parsed head test failure	1,774	26.94%	1.07%
Resolved base run found	1,348	75.99%	0.82%
Known base failure set	897	66.54%	0.54%
Emitted strict benchmark instances	762	84.95%	0.46%

TABLE III
BREAKDOWN BY SPLIT AND PROGRAMMING LANGUAGE

Split	Inst.	Labels	Tests	Java Inst.	Python Inst.
all	897	4,384	2,500	751 (83.7%)	146 (16.3%)
relaxed	770	4,214	2,484	645 (83.8%)	125 (16.2%)
strict	762	4,168	2,466	638 (83.7%)	124 (16.3%)

(docs/DATASHEET.md §4). Failure messages were scanned for credentials; zero blocking secrets were identified, while 5 review samples were flagged for human verification.

IV. DATASET OVERVIEW AND QUALITY

A. Corpus and Attrition Funnel

The progressive filtering of repositories and workflow runs is summarized in Table II. Of the 165,349 workflow runs swept across 76 repositories, 12,581 (7.61%) concluded with a failure status. Among these, 6,586 had raw job logs preserved on disk, and 1,774 parsed to at least one test failure. Base-run matching resolved 1,348 of these runs, of which 897 possessed a known base failure set (either via an `exact_green` run or a parsed base log). Filtering out flakiness and runs lacking test output yielded 762 qualified benchmark instances in the strict split.

B. Language and Split Distribution

Table III summarizes the benchmark splits. The `strict` benchmark split comprises 762 pull-request instances and 4,168 fault-revealing labels representing 2,466 distinct tests across 37 repositories. Java accounts for 83.73% of instances (638/762) and 92.49% of labels (3,855/4,168), while Python represents 16.27% of instances (124/762) and 7.51% of labels (313/4,168).

C. Test Binding and Data Quality

Table IV reports test binding and error classification. Across 5,985 evaluated (`repo`, `test_id`) pairs, 94.29% (5,643/5,985) bind successfully to source files. Full-confidence FQCN resolution accounts for 63.83% (3,820/5,985), while basename matching resolves 30.46% (1,823/5,985). Only 5.71% remain unbound (2.97% not found, 2.74% ambiguous).

An automated regex audit classified failure messages into four classes: code-level assertions accounted for 17.49%

TABLE IV
TEST-TO-FILE BINDING AND QUALITY INDICATORS

Binding Strategy / Quality Metric	Count	Percentage
Full-confidence FQCN match ($c = 1.0$)	3,820	63.83%
Basename-only match ($c = 0.5$)	1,823	30.46%
Unbound: not found in source tree	178	2.97%
Unbound: ambiguous multi-match	164	2.74%
Combined exact binding	5,643	94.29%
Code-level assertions (regex lower bound)	729	17.49%
Timeout failures	112	2.69%
Strict environment failures (GPU, network)	89	2.14%
Unknown error text / unparsed stack trace	3,238	77.69%

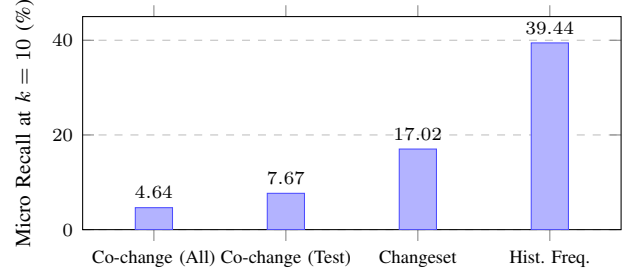


Fig. 2. Micro recall at $k = 10$ across test selection proxies evaluated on BR-BENCH ($n = 576$ identical instances, 1,316 positive labels).

(729/4,168), timeout errors represented 2.69% (112/4,168), and strict environment failures (e.g., missing CUDA GPUs or network timeouts) accounted for 2.14% (89/4,168). A majority of labels (77.69%, 3,238/4,168) remain unknown due to uncaptured stack traces or unparsed error texts.

V. HOW THE DATA HAS BEEN USED: RQ1 EXAMPLE

To demonstrate how BR-BENCH serves as an empirical benchmark, we evaluate an example research question (RQ1): *How accurately do conventional change impact analysis proxies and historical baselines identify fault-revealing tests?*

A. Experimental Setup and Leakage Audit

We evaluate four test selection strategies at a budget of $k = 10$ candidate files (candidates are files):

- 1) *Co-change (All)*: Trailing 365-day co-change partners mined from Git commits.
- 2) *Co-change (Test Only)*: Co-change restricted to conventional test file patterns (`src/test/`, `*Test.java`, `test_*.py`).
- 3) *Changeset Baseline*: The pull request’s changed files themselves.
- 4) *Historical Frequency*: The k most frequently failing bound test files from strictly earlier runs (strict labels, each (test, instance) counted once).

To prevent temporal data leakage, co-change history was mined strictly prior to each run’s start timestamp. An audit revealed that a naive static co-change table leaked future commits in 713/713 audited instances, and an uncorrected historical baseline double-counted failures across multiple splits (224,758 rows for 78,399 distinct labels). After enforcing per-instance trailing windows, symmetric partner lookups, and single-split counting, zero leakage violations remain across all 762 instances.

B. Results

All methods are evaluated on the identical set of $n = 576$ instances where the co-change proxy yields predictions (1,316 positive test labels; 462 Java, 114 Python). As shown in Fig. 2, all-partner co-change achieves a micro recall of only 4.64% (61/1,316; mean P/R/J: 0.011 / 0.051 / 0.010). Restricting co-change to conventional test files improves micro recall to 7.67% (101/1,316; mean P/R/J: 0.054 / 0.087 / 0.038). The changeset baseline recalls 17.02% (224/1,316; mean P/R/J: 0.039 / 0.245 / 0.037). In contrast, historical failure frequency achieves 39.44% micro recall (519/1,316; mean P/R/J: 0.185 / 0.517 / 0.178). Removing environment and timeout failure labels leaves this ordering unchanged. This evaluation demonstrates that historical test failure frequency dramatically outperforms version-control co-change heuristics at identifying true regressions.

VI. ORIGINALITY AND SIMILAR DATASETS

BR-BENCH differs substantively from existing benchmarks:

- **RTPTorrent** [7]: While RTPTorrent provides test outcomes from Travis CI for Java, it organizes data by build job rather than pull-request diffs, predates GitHub Actions, and focuses on test prioritization. BR-BENCH targets diff-to-failing-test prediction across Java and Python.
- **GHALogs** [8]: GHALogs provides an archive of raw GitHub Actions logs. However, it does not extract test-level outcomes, link pull-request diffs, resolve base commits, or detect flaky tests. BR-BENCH supplies these processed, fault-revealing labels.
- **Static and Dynamic RTS Tools**: Tools such as Ekstazi [3], STARTS [4], and Chianti [12] select tests reaching modified code dynamically or statically. These tools compute safe reachability over-approximations, whereas BR-BENCH captures empirical failures during CI runs.
- **Industrial Studies**: Industrial studies from Meta [5] deployed predictive models, while Google [6] analyzed large-scale continuous testing workloads. BR-BENCH provides an open dataset enabling the research community to replicate and benchmark predictive test selection techniques on open-source repositories.

VII. LIMITATIONS AND THREATS

We identify several limitations and threats to validity:

- 1) *Roadmap Sample Size Gates*: Our roadmap Gate 1 threshold ($\geq 5,000$ strict positives) was **not met**, reaching 4,168/5,000 labels (83.36%) and 762/5,000 instances (15.24%). Furthermore, Gate 1.5 ($\geq 70\%$ test binding) was met on the combined metric (94.29%, 5,643/5,985) but **not met** at full-confidence FQCN resolution (63.83%, 3,820/5,985).
- 2) *Parser Evaluation*: Parser evaluation figures on our fixture corpus (46/46) and holdout v1 (30/30) were derived from development sets used during parser tuning. Independent parser precision on holdout v5 was **not measured** [MISSING: blind human annotations on

holdout v5 fixtures] due to the lack of completed blind human annotations.

- 3) *Observational Ground Truth*: Labels represent observational CI execution outcomes rather than causal defect injections or isolated re-executions. Non-test environmental failures remain present in a small minority of labels (2.14% strict environment, 2.69% timeout).
- 4) *90-Day Retention Cliff*: Because GitHub Actions purges raw log artifacts after 90 days, 33.21% of failed runs (4,458/13,422) had already expired at the time of harvesting across the 77-repository harvest corpus (as of the last capture, 2026-08-29). Exact reproduction must rely on our archived raw store rather than live GitHub API calls.
- 5) *Repository Skew*: The strict benchmark split is dominated by Java repositories (35 Java vs. 2 Python), with high volume from a small set of active projects (e.g., Apache Beam, Stirling-PDF).

VIII. FUTURE RESEARCH QUESTIONS AND IMPROVEMENTS

BR-BENCH opens promising directions for empirical software engineering research, including: (1) *Machine Learning for Test Selection*: training graph neural networks and language models on code diffs and test paths to predict regressions without execution overhead; (2) *Independent Test Re-execution*: executing a planned causal subset in isolated containerized environments to distinguish true regressions from environmental flakes; and (3) *Multi-Language Expansion*: extending coverage to JavaScript, TypeScript, and Go repositories to balance the language distribution.

IX. AVAILABILITY AND REPRODUCIBILITY

All datasets and extraction scripts are released under open licenses [LICENCE: CC-BY 4.0 planned, confirm]. The release bundle deposited on Zenodo at [DOI-DATA] holds the processed `instances`, `outcomes`, and `cochange` tables (165,349 / 12,766 / 175,204 rows), NOT the raw log store. The complete source code pipeline is available on GitHub at [DOI-CODE]. The dataset numbers in this paper can be regenerated offline in approximately 8 minutes on a standard laptop by executing `make tables`, and the test suite comprising 782 automated tests (782 passed, 1 skipped) passes cleanly via `make test`.

X. CONCLUSION

We presented BR-BENCH, an open benchmark linking pull-request modifications to fault-revealing tests across Java and Python GitHub Actions pipelines. By resolving base runs, filtering same-commit flakiness, and binding tests to source files, BR-BENCH provides an empirical ground truth for regression test selection and impact analysis. Our initial evaluation demonstrates that historical test failure frequency substantially outperforms conventional co-change proxies, establishing a foundation for future predictive test selection research.

REFERENCES

- [1] M. Borg, K. Wnuk, B. Regnell, and P. Runeson, “Supporting change impact analysis using a recommendation system: An industrial case study in a safety-critical context,” *IEEE Trans. Softw. Eng.*, vol. 43, no. 7, pp. 675–700, 2017.
- [2] Y. Huang, J. Jiang, X. Luo, X. Chen, Z. Zheng, N. Jia, and G. Huang, “Change-patterns mapping: A boosting way for change impact analysis,” *IEEE Trans. Softw. Eng.*, vol. 48, no. 7, pp. 2376–2398, 2022.
- [3] M. Gligoric, L. Eloussi, and D. Marinov, “Practical regression test selection with dynamic file dependencies,” in *Proc. 2015 Int. Symp. Softw. Test. Anal. (ISSTA)*. ACM, 2015, pp. 211–222.
- [4] O. Legunsen, A. Shi, and D. Marinov, “STARTS: STatic regression test selection,” in *Proc. 32nd IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE), Tool Demos*. IEEE, 2017, pp. 949–954.
- [5] M. Machalica, A. Samykin, M. Porth, and S. Chandra, “Predictive test selection,” in *Proc. 41st Int. Conf. Softw. Eng.: Softw. Eng. Pract. (ICSE-SEIP)*. IEEE, 2019, pp. 91–100.
- [6] Memon, Gao, Nguyen, Dhanda, Nickell, Siemborski, and Micco, “Taming Google-scale continuous testing,” in *Proc. 39th Int. Conf. Softw. Eng.: Softw. Eng. Pract. (ICSE-SEIP)*. IEEE, 2017, pp. 233–242.
- [7] T. Mattis, P. Rein, F. Dürsch, and R. Hirschfeld, “RTPTorrent: An open-source dataset for evaluating regression test prioritization,” in *Proc. 17th Int. Conf. Mining Softw. Repos. (MSR)*. ACM, 2020, pp. 385–396.
- [8] F. Moriconi, T. Durieux, J.-R. Falleri, R. Troncy, and A. Francillon, “GHALogs: Large-scale dataset of GitHub actions runs,” in *Proc. 22nd Int. Conf. Mining Softw. Repos. (MSR), Data and Tool Showcase Track*. IEEE, 2025.
- [9] O. Dabic, E. Aghajani, and G. Bavota, “Sampling projects in GitHub for MSR studies,” in *Proc. 18th Int. Conf. Mining Softw. Repos. (MSR)*. IEEE, 2021, pp. 560–564.
- [10] J. Bell, O. Legunsen, M. Hilton, L. Eloussi, T. Yung, and D. Marinov, “DeFlaker: Automatically detecting flaky tests,” in *Proc. 40th Int. Conf. Softw. Eng. (ICSE)*. ACM, 2018, pp. 433–444.
- [11] T. Gebru, J. Morgenstern, B. Vecchione, J. W. Vaughan, H. Wallach, H. Daumé III, and K. Crawford, “Datasheets for datasets,” *Commun. ACM*, vol. 64, no. 12, pp. 86–92, 2021.
- [12] X. Ren, F. Shah, F. Tip, B. G. Ryder, and O. Chesley, “Chianti: A tool for change impact analysis of Java programs,” in *Proc. 19th ACM SIGPLAN Conf. Object-Oriented Program., Syst., Lang., Appl. (OOPSLA)*. ACM, 2004, pp. 432–448.